

Détection & Vérification de l'État Réseau

- Analyse technique d'une implémentation de connectivité mobile avec le package `connectivity_plus`
- Présenter par :
- **Groupe 7**

Qu'est-ce que Connectivity_plus 6.0.0 ?

Définition :

Connectivity_plus est un plugin officiel de la communauté *Flutter Community* qui permet de découvrir l'état de la connectivité réseau (Wi-Fi, données mobiles, Ethernet, etc.) sur plusieurs plateformes (Android, iOS, Web, macOS).

NB : La version actuelle est le : 7.3.0

La Révolution de la Version 6.0.0 :

Avant la v6.0.0 : La méthode

checkConnectivity() renvoyait une valeur unique (Ex: ConnectivityResult.wifi).

Depuis la v6.0.0 : Elle renvoie désormais une **Liste** de résultats (List<ConnectivityResult>).

Pourquoi ce changement ? Les smartphones modernes peuvent être connectés à plusieurs réseaux en même temps (ex: Wi-Fi actif + Données mobiles en arrière-plan). L'application doit donc analyser la liste complète.

Introduction & Installation du Package

Objectif de l'application :

Permettre à une application mobile de connaître à tout moment son état de connexion afin d'éviter les erreurs de chargement de données pour l'utilisateur.

Installation du plugin :

Pour ajouter la gestion réseau à notre projet Flutter, nous utilisons la commande officielle dans le terminal à la racine du projet :

Bash

```
flutter pub add connectivity_plus
```

```
dependencies:  
  flutter:  
    sdk: flutter  
  connectivity_plus: ^7.3.0
```

Télécharge la dernière version stable du package.

Inscrit automatiquement la dépendance dans le fichier de configuration pubspec.yaml.

Résultat

Dans le fichier:
pubspec.yaml

Ligne(s) de code	Élément / Commande	Explication Pédagogique Détaillée
<code>import 'package:flutter/material.dart';</code>	Importation Material	Importe la bibliothèque graphique officielle de Google pour utiliser les composants d'interface (boutons, textes, thèmes).
<code>import 'package:connectivity_plus/...';</code>	Importation Plugin	Importe le package externe pré-installé avec flutter pub add permettant de communiquer avec la carte réseau du téléphone.
<code>void main() { runApp(...); }</code>	Point d'entrée	La fonction principale qui lance l'application. runApp prend le widget racine (MyApp) et l'affiche sur tout l'écran.
<code>class MyApp extends StatelessWidget</code>	Configuration de l'App	Widget racine immuable (statique). Il sert à configurer les paramètres globaux (titre, thèmes, masquage de la bannière "DEBUG").
<code>MaterialApp(...)</code>	Widget MaterialApp	Le conteneur principal obligatoire pour une application au style Material Design. Il définit la page d'accueil avec sa propriété home.
<code>class NetworkPage extends StatefulWidget</code>	Widget d'Écran Dynamique	Déclare que cette page possède un état fluide (les données affichées à l'écran pourront changer au cours du temps).
<code>String message = "...";</code>	Variable d'État	Variable de type texte qui stocke le message actuel affiché à l'utilisateur. Sa valeur change selon la connexion.
<code>Future<void></code>	Type de retour Future	Indique que la fonction effectue une opération asynchrone (une tâche de fond qui prend du temps et renverra un résultat "dans le futur").

async	Marqueur Asynchrone	Mot-clé qui permet d'utiliser le mot-clé await à l'intérieur de la fonction sans bloquer l'affichage de l'application.
await Connectivity().check...	Interrogation Réseau	await suspend l'exécution de la fonction le temps que la puce réseau (Wi-Fi/4G) du téléphone renvoie l'état actuel sous forme de liste.
final result = ...;	Stockage du Résultat	Stocke la liste des types de connexions actives détectées par l'appareil dans une variable nommée result.
result.contains(...)	Nouveauté v6.0.0	Spécifique à la v6.0.0. Comme le résultat est désormais une List, on utilise .contains() pour vérifier si le statut "aucune connexion" (none) est présent dans la liste.
setState(() { ... });	Mise à jour de l'UI	Commande cruciale de Flutter. Elle modifie la variable message et force l'écran à se redessiner immédiatement pour afficher la mise à jour.
Scaffold(...)	Squelette de la page	Fournit la structure de base de l'écran (la barre du haut appBar et la zone centrale body).
appBar: AppBar(...)	Barre Supérieure	Affiche l'en-tête de la page avec le titre de votre projet textuel.
Column(...)	Mise en page verticale	Organise les widgets enfants (Text et ElevatedButton) les uns en dessous des autres verticalement au centre de l'écran.
ElevatedButton(...)	Bouton d'Action	Un bouton Material Design surélevé avec une ombre. Sa propriété onPressed appelle la fonction verifierConnexion au clic.

